

北京交通大学软件学院

北京交通大学考试试题 (A 卷)

课程名称: 数据结构 学年学期: 2024-2025 学年第 1 学期

课程编号: M310002B 开课学院: 软件学院 出题教师: 数据结构课程组

学生姓名: _____ 学号: _____ 任课教师: _____ 班级: _____

一、选择题 (共 20 分, 每题 2 分)

1. 以下算法的时间复杂度为 ()。

```
void fun(int n) {  
    int i = 1;  
    while (i <= n) i = i * 2;  
}
```

- A. $O(n)$ B. $O(n^2)$ C. $O(n\log_2 n)$ D. $O(\log_2 n)$

2. 在一个单链表中, 已知 q 所指结点是 p 所指结点的前驱结点, 若在 q 和 p 之间插入结点 s , 则执行 ()。

- A. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
B. $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p;$
C. $q \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
D. $p \rightarrow \text{next} = s; s \rightarrow \text{next} = q;$

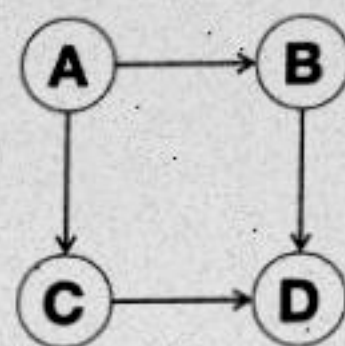
3. 设 a, b, c, d, e, f 以所给的次序进栈, 若在进栈操作时, 允许出栈操作, 则下面得不到的序列为 ()。

- A. $fedcba$ B. $bcaefd$ C. $dcefba$ D. $cabdef$

4. 在矩阵压缩存储中, 用三元组 (i, j, value) 表示稀疏矩阵, 其中 i 表示行号, j 表示列号, value 表示矩阵中的非零元素。以下关于压缩三元组的描述中, 正确的是 ()。

- A. 压缩三元组的行号 i 和列号 j 必须按降序排列。
B. 对于一个 $m \times n$ 的稀疏矩阵, 压缩三元组的存储空间与 $m \times n$ 成正比。

- C. 压缩三元组在存储时只包含非零元素的位置信息及其值。
D. 压缩三元组适用于所有矩阵类型, 包括密集矩阵。
5. 假设一棵二叉树的结点个数为 50, 则它的最小高度是 ()。
A. 4 B. 5 C. 6 D. 7
6. 已知某二叉树的先序遍历序列为 ABDCE, 中序遍历序列: DBACE。那么该二叉树的后序遍历序列为 ()。
A. DBECA B. DBEAC C. DEBCA D. DBCEA
7. 用邻接表存储的图的深度优先遍历算法类似于树的 ()。
A. 中序遍历 B. 先序遍历
C. 后序遍历 D. 按层次遍历
8. 如右图所示的有向无环图的拓扑排序有 () 种可能。
A. 1 B. 2 C. 3 D. 4
9. 采用哈夫曼编码对这些字符进行编码, 以下描述正确的是 ()。
A. 哈夫曼树中频率最高的字符一定是根节点。
B. 哈夫曼编码总是等长编码, 字符越多, 编码越长。
C. 哈夫曼树的构造过程需要每次选出频率最小的两个节点合并。
D. 哈夫曼编码不适用于压缩数据, 因为它不能减少存储空间。
10. 对于序列 {6, 3, 8, 5, 2, 7, 4, 1}, 使用快速排序算法升序排序, 以第一个元素作为基准值。经过第一趟排序后, 序列变为 ()。
A. {3, 2, 1, 4, 6, 8, 7, 5}
B. {1, 3, 4, 5, 2, 6, 7, 8}
C. {1, 3, 2, 4, 6, 7, 5, 8}
D. {1, 4, 3, 2, 5, 6, 8, 7}



二、判断题 (共 10 分, 每题 1 分)

1. 在存储数据时, 通常不仅要存储各数据元素的值, 而且要存储数据元素之间的关系。

2. 循环队列判空的条件是 $\text{front} == \text{rear}$, 判满的条件是 $(\text{rear} + 1) \% \text{MAXSIZE} == \text{front}$ 。
3. 队列的“先进先出”特性是指每当有删除操作时, 总要先做一次插入操作。
4. KMP 算法通过计算模式串的 next 数组 (或部分匹配表) 来确定当出现不匹配时模式串应该移动的位置, 而主串的指针则保持不动或者继续向后移动, 不会变小。
5. 平衡二叉树的中序遍历结果不一定是一个有序序列。
6. 在 Prim 算法中, 起始节点的选择会影响最终生成树的权值总和。
7. 存储无向图的邻接矩阵是对称的, 故只需存储邻接矩阵的下 (或上) 三角部分。
8. 已知一个广义表 $(1, (2, 3), ((4), 5, ()), (((6))))$, 则该广义表的深度为 3。
9. 在一棵先序线索二叉树中, 所有右指针一定指向它的先序后继节点。
10. 快速排序和堆排序都是不稳定的排序算法。

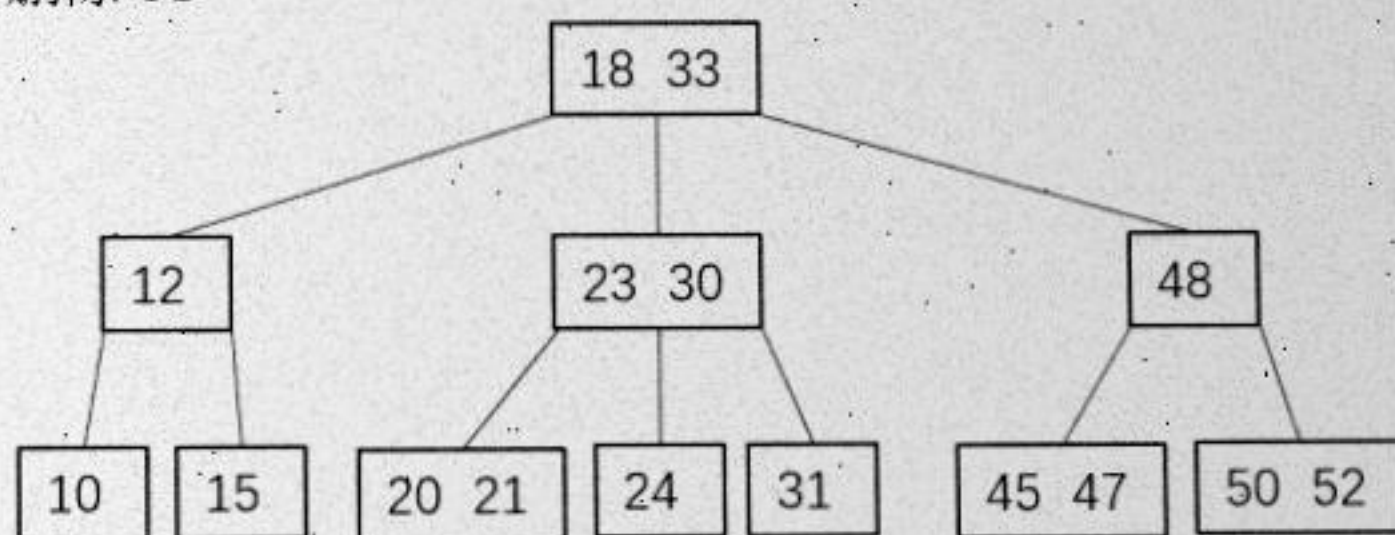
三、综合分析题 (共 50 分)

1. (4 分) 已知一棵完全二叉树共有 892 个结点, 试求:
 - (1) 该树的高度
 - (2) 单分支结点数
 - (3) 叶子结点数
 - (4) 最后一个分支结点的序号
2. (10 分) 某个工程各个活动之间的关系和各个活动所需的时间如下表所示 (其中“-”表示无先序活动), 请回答以下问题:

活动 (边)	a	b	c	d	e	f	g	h	i	j
所需时间	2	4	1	3	4	2	1	6	2	5
先序活动	-	-	a	a	a	d	d	b, c	e, f	h, g

- (1) 画出对应的 AOE 网 (事件编号用数字表示)
- (2) 计算每个事件最早开始和最晚开始的时间
- (3) 求出关键路径

3. (10 分) 有一棵二叉排序树, 按先序遍历得到的序列为 {12, 5, 2, 8, 6, 10, 16, 15, 18, 20}, 回答以下问题:
- (1) 画出该二叉排序树
 - (2) 求等概率下查找成功和查找不成功的平均查找长度
 - (3) 画出删除结点 12 之后的二叉排序树
4. (10 分) 给出关键字序列 {15, 18, 29, 12, 35, 32, 27, 23, 10, 20}, 对该序列进行升序排序:
- (1) 写出对该序列进行希尔排序的每一趟序列状态, 其中希尔排序的增量序列为 5, 2, 1
 - (2) 画出采用堆排序时建立的初始堆, 以及第一次输出堆顶元素后筛选调整以后的堆
5. (10 分) 将关键字序列 {17, 5, 7, 40, 34, 37, 15, 30, 26, 23, 3, 33} 插入哈希表 H 中。已知哈希函数 $H(k) = k \% 13$, 散列地址空间为 0-12, 按线性探测再散列解决冲突的方法构造哈希表。写出构造后的哈希表, 并求出等概率下查找成功和失败时的平均查找长度。
6. (6 分) 对下图的 3 阶 B-树依次执行下列操作, 画出每步的操作结果。(后一步操作是在前几步操作结果的基础上进行操作)
- (1) 插入 55
 - (2) 删除 24
 - (3) 删除 31



四、算法题 (共 20 分)

1. (10 分) 下列算法是对冒泡排序算法的改造, 实现双向冒泡排序, 填写程序中的空白处实现上述功能。(双向冒泡排序指的是每一趟通过两个相邻关键字比较产生最小和最大的元素)。

```
#include <stdio.h>
```

```

#define MAXSIZE 1000
typedef struct {
    int    key;           // 关键字项
    int    otherinfo;     // 其它数据项
} RcdType;
RecType  r[maxsize+1];   // r[0]闲置
void  dbbubble( RcdType  r,int n)
{
    int  i=1, j,b=1;
    RcdType t;
    while (b)
    {
        b=0;
        for (j=n-i+1; j>=i+1; j--)
            if ( (1) ) {
                b=1;
                t=r[j];
                r[j]=r[j-1];
                (2)
            }
        for ( (3) )
            if (r[j].key> r[j+1].key)
            {
                b=1;
                t=r[j];
                (4)
                r[j+1]=t;
            }
        (5)
    }
}

```

2. (10 分) 用 C 语言描述树的孩子兄弟链表结构, 编写程序求树的高度。

结构定义如下:

```

typedef struct CSNode
{
    char  data;
    struct CSNode *fch, *nsib;
}

```

```
    } CSNode, *CSTree;
```

函数原型为:

```
int TreeDepth(CSTree T)
{
    //补充完整
}
```